For Spring Boot applications, you can leverage the Actuator library to expose information about their health through a `/actuator/health` HTTP endpoint, including details about the application's status and the components in use, like databases, event brokers, and config servers.

Spring Boot Actuator is a useful library, providing many endpoints for monitoring and managing Spring Boot applications. Such endpoints can be exposed through HTTP or JMX, but either way we must protect them from unauthorized access. We'll limit ourselves to using the HTTP endpoints, so we can use Spring Security to define access policies like those for any other endpoint we've worked with so far.

This section will cover configuring health endpoints in Spring Boot applications using Actuator. You'll then see how you can define liveness and readiness probes so Kubernetes can use its self-healing functionality.

13.2.1 Defining health probes for Spring Boot applications using Actuator

First of all, open the build.gradle file in the Catalog Service project (catalog-service), and ensure that it contains a dependency on Spring Boot Actuator (we used it in chapter 4 for refreshing configuration at runtime).

Listing 13.5 Adding dependency for Spring Boot Actuator in Catalog Service

```
dependencies {
  ...
  implementation 'org.springframework.boot:spring-boot-starter-actuator'
}
```

There are a few viable solutions for protecting the Spring Boot Actuator endpoints. For example, you could enable HTTP Basic authentication just